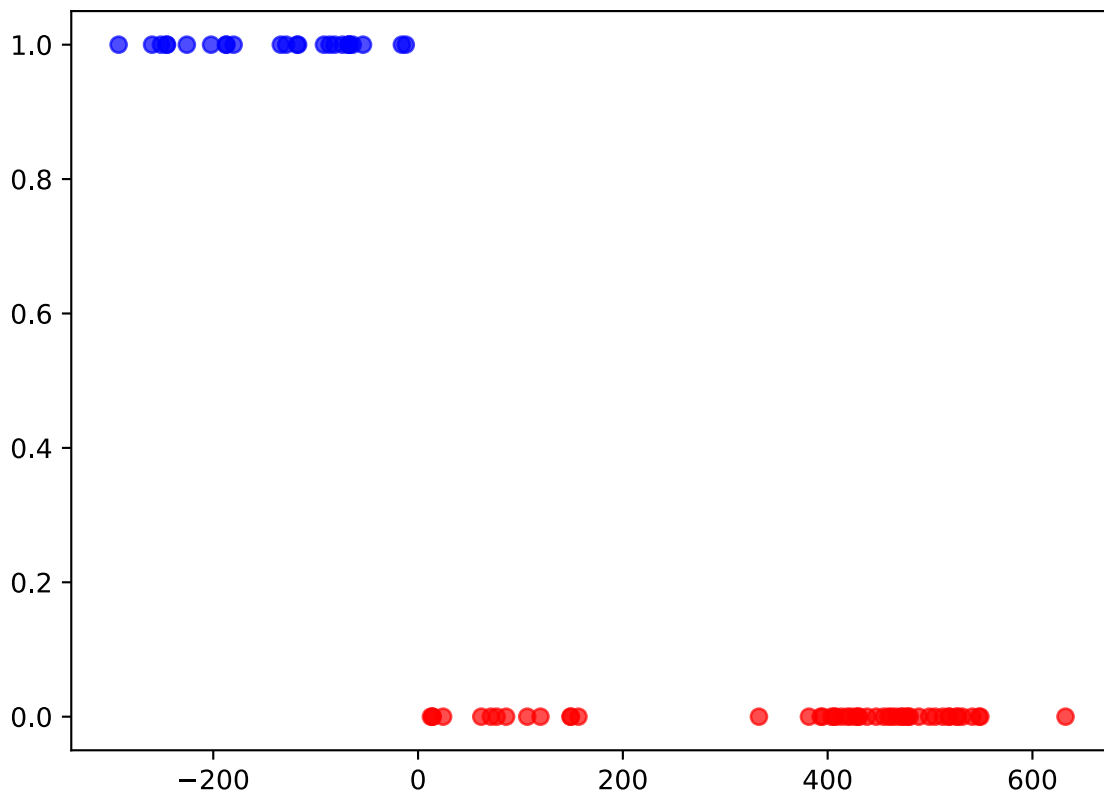


## Exercise Set 2

### Problem 8

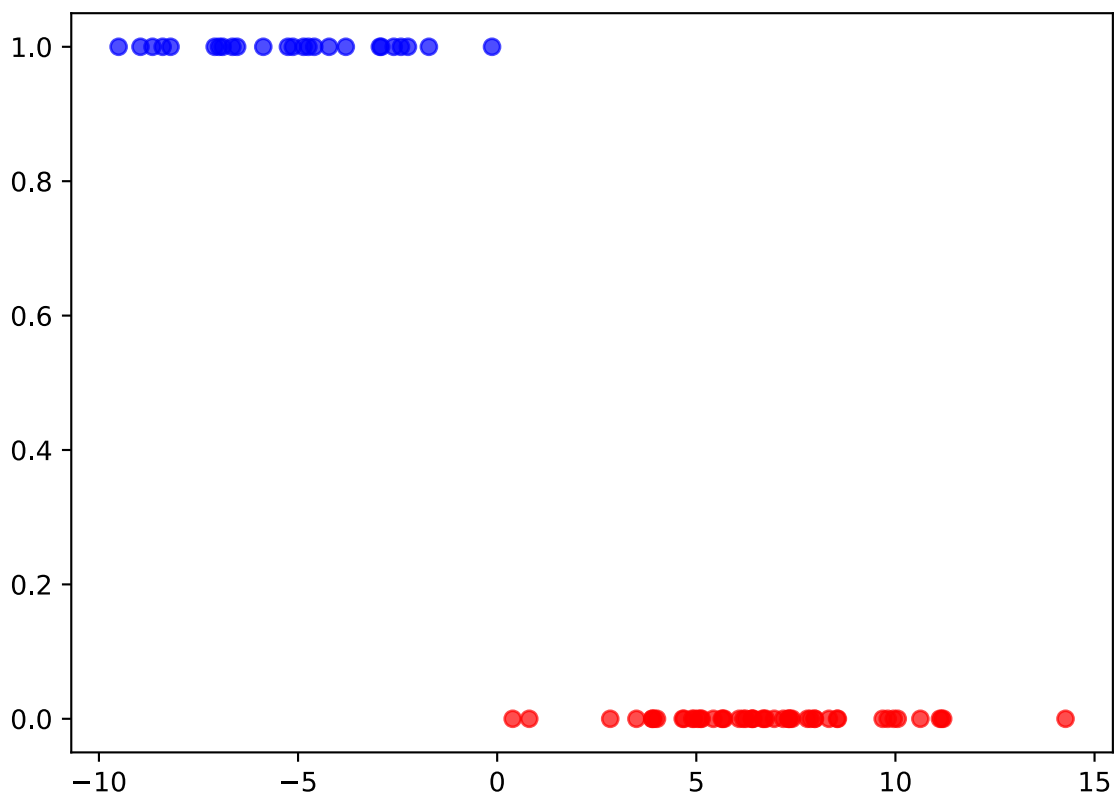
#### Task a

```
Test Accuracy: 0.9333333333333333
Train Accuracy: 1.0
Coefficients: [[ 2.14948051e+01 -8.18056613e+01  1.97487470e+00
 4.78735619e-02]]
```



#### Task b

```
Test Accuracy: 0.9733333333333334
Train Accuracy: 1.0
Coefficients: [[ 1.01931750e+00 -1.77159746e+00 -3.69652638e-02
 -1.29931343e-03]]
```



### Task c

The coefficients are large because the model is trying to push probabilities to 0 or 1. Coefficients:  
[[ 2.14948051e+01 -8.18056613e+01 1.97487470e+00 4.78735619e-02]]

### Problem 9

Claim

For two classes and one feature, QDA is not linear if  $\sigma_1^2 \neq \sigma_2^2$ .

---


$$x \mid y = k \sim \mathcal{N}(\mu_k, \sigma_k^2), \quad k = 1, 2.$$

$$p(y = 1 \mid x) > p(y = 2 \mid x) \Leftrightarrow \pi_1 f_1(x) > \pi_2 f_2(x),$$

where  $\pi_k$  are class priors and

$$f_k(x) = \frac{1}{\sqrt{2\pi}\sigma_k} \exp\left(-\frac{(x - \mu_k)^2}{2\sigma_k^2}\right).$$

Thus, classify as class 1 if

$$\pi_1 \frac{1}{\sigma_1} \exp\left(-\frac{(x - \mu_1)^2}{2\sigma_1^2}\right) > \pi_2 \frac{1}{\sigma_2} \exp\left(-\frac{(x - \mu_2)^2}{2\sigma_2^2}\right).$$

Taking natural logs:

$$\log \pi_1 - \log \sigma_1 - \frac{(x - \mu_1)^2}{2\sigma_1^2} > \log \pi_2 - \log \sigma_2 - \frac{(x - \mu_2)^2}{2\sigma_2^2}.$$

Rearranging:

$$\frac{(x - \mu_2)^2}{2\sigma_2^2} + \frac{(x - \mu_1)^2}{2\sigma_1^2} > \log \pi_2 - \log \pi_1 + \log \sigma_1 - \log \sigma_2.$$

Expanding both squares:

$$\frac{x^2 - 2\mu_2 x + \mu_2^2}{2\sigma_2^2} > \frac{x^2 - 2\mu_1 x + \mu_1^2}{2\sigma_1^2}$$

for some constant C. Collecting terms in x:

$$x^2 \left( \frac{1}{2\sigma_2^2} - \frac{1}{2\sigma_1^2} \right) + x \left( \frac{\mu_1}{\sigma_1^2} - \frac{\mu_2}{\sigma_2^2} \right) + \left( \frac{\mu_2^2}{2\sigma_2^2} - \frac{\mu_1^2}{2\sigma_1^2} \right) > C.$$

Because

$$\sigma_1^2 \neq \sigma_2^2 \Rightarrow \frac{1}{2\sigma_2^2} - \frac{1}{2\sigma_1^2} \neq 0,$$

there is a nonzero coefficient for  $x^2$ . Therefore, the boundary is quadratic, not linear.

---

Conclusion

QDA is not linear when  $\sigma_1^2 \neq \sigma_2^2$ .

## Problem 10

### Task a

```
Class 1 Mean:
  bill_length_mm      38.124
  bill_depth_mm       18.336
  flipper_length_mm   188.880
  body_mass_g         3576.000
  dtype: float64
----
Class 0 Mean:
```

```

    bill_length_mm      47.818
    bill_depth_mm       15.890
    flipper_length_mm   211.300
    body_mass_g         4657.000
    dtype: float64
----
Class 1 Std:
    bill_length_mm      2.781528
    bill_depth_mm       1.204118
    flipper_length_mm    6.320074
    body_mass_g         461.343148
    dtype: float64
----
Class 0 Std:
    bill_length_mm      3.599472
    bill_depth_mm       1.965441
    flipper_length_mm   11.792855
    body_mass_g         787.531017
    dtype: float64
----

```

Prior 1 - Adelie, Prior 0 - notAdelie

```

Prior 1: 0.33766233766233766
Prior 0: 0.6623376623376623

```

## Task b

$$\begin{aligned}
 P(A | X) &= \frac{P(X | A) P(A)}{P(X)} \\
 &= \frac{P(\text{bill\_length} | A) P(\text{bill\_depth} | A) P(\text{flipper\_length} | A) P(\text{body\_mass} | A) P(A)}{P(X)} \\
 &= \frac{\left[ \prod_{j=1}^4 \mathcal{N}(x_j | \mu_{A,j}, \sigma_{A,j}) \right] P(A)}{P(X)}
 \end{aligned}$$

## Task c

```

def P_class_given_X(X, class_mean, class_std, prior, class_index):
    mu = class_mean[class_index]
    sigma = class_std[class_index]
    coeff = 1 / (np.sqrt(2 * np.pi) * sigma)
    exponent = np.exp(-0.5 * ((X - mu) / sigma) ** 2)
    P_X_given_class = np.prod(coeff * exponent, axis=1)
    return prior[class_index] * P_X_given_class

```

```
def compute_probabilities(X, class_mean, class_std, prior):
    P_0 = P_class_given_X(X, class_mean, class_std, prior, 0)
    P_1 = P_class_given_X(X, class_mean, class_std, prior, 1)
    total = P_0 + P_1
    return P_0 / total, P_1 / total

probs_adelie = compute_probabilities(X_test, class_mean, class_std, prior)[1]
y_pred = (probs_adelie >= 0.5).astype(int)
y_pred = np.where(y_pred == 1, "Adelie", "notAdelie")
```

Classification accuracy on test set: 0.92

## Problem 11

### Task a

Discriminative learning is better according to the authors

“Discriminative classifiers are almost always to be preferred”

### Task b

“parametric family of probabilistic models” - a family of distributions  $p(x,y)$  that can be described by a finite set of parameters.

- The family where inputs are discrete (so joint  $p(x,y)$  is modeled via a multinomial / categorical model) – corresponding to discrete-feature classification.
- The family where inputs are continuous and (in their analysis) assumed to have Gaussian class-conditional distributions with diagonal covariance (or more generally, normal distributions).

Correspondingly:

- $h_{\text{Gen}}$  is the classifier obtained by fitting the joint model  $p(x,y)$  by maximizing the joint (log)-likelihood of data under that parametric family.
- $h_{\text{Dis}}$  is the classifier obtained by fitting the conditional model  $p(y | x)$  using the same parametric family – i.e. the discriminative analogue

### Task c

According to the last paragraph of the Introduction, Figure 1 suggests that as the number of training examples grows, error curves for the generative and discriminative classifiers cross: for small sample sizes the generative classifier (naive Bayes) has lower error, but as sample size increases the discriminative classifier (logistic regression) overtakes and has lower error.

## Problem 12

### Task a

$$p(y = 1 \mid x_1, x_2) = \sigma(0.1 - 2x_1 + x_2 + 0.2x_1x_2)$$

When generating probabilities there exists an interaction term between features  $x_1$  and  $x_2$ :  $0.2x_1x_2$ . Therefore NB assumption of feature independence is violated.

### Task b

```
def optimal_bayes_probs(X):  
    linear_combination = 0.1 - 2 * X["x1"] + X["x2"] + 0.2 * X["x1"] * X["x2"]  
    return 1 / (1 + np.exp(-linear_combination))  
  
def optimal_bayes_classifier(X):  
    probs = optimal_bayes_probs(X)  
    return (probs >= 0.5).astype(int)
```

NB - sklearn.naive\_bayes.GaussianNB().

LR - sklearn.linear\_model.LogisticRegression().

LRi - sklearn.linear\_model.LogisticRegression() with interaction term.

Dummy Classifier - sklearn.dummy.DummyClassifier(strategy="stratified").

Accuracy:

|      | NB     | LR     | LRi    | OptimalBayes | Dummy  |
|------|--------|--------|--------|--------------|--------|
| 8    | 0.6643 | 0.6964 | 0.6779 | 0.793        | 0.4989 |
| 16   | 0.7635 | 0.7574 | 0.7608 | 0.793        | 0.5145 |
| 32   | 0.7476 | 0.7854 | 0.7853 | 0.793        | 0.4992 |
| 64   | 0.7855 | 0.7917 | 0.7917 | 0.793        | 0.4945 |
| 128  | 0.7889 | 0.7909 | 0.7813 | 0.793        | 0.4958 |
| 256  | 0.7919 | 0.7912 | 0.7925 | 0.793        | 0.5093 |
| 512  | 0.7903 | 0.7905 | 0.7930 | 0.793        | 0.5043 |
| 1024 | 0.7919 | 0.7921 | 0.7917 | 0.793        | 0.4918 |
| 2048 | 0.7888 | 0.7915 | 0.7921 | 0.793        | 0.5088 |
| 4096 | 0.7902 | 0.7915 | 0.7924 | 0.793        | 0.5031 |

Perplexity:

|      | NB       | LR       | LRi      | OptimalBayes | Dummy        |
|------|----------|----------|----------|--------------|--------------|
| 8    | 3.440829 | 1.746579 | 1.772886 | 1.546456     | 6.982305e+07 |
| 16   | 1.783428 | 1.624282 | 1.619802 | 1.546456     | 3.979269e+07 |
| 32   | 1.708581 | 1.635378 | 1.635401 | 1.546456     | 6.907211e+07 |
| 64   | 1.597867 | 1.582407 | 1.580771 | 1.546456     | 8.182282e+07 |
| 128  | 1.559905 | 1.560774 | 1.597654 | 1.546456     | 7.807731e+07 |
| 256  | 1.555375 | 1.550317 | 1.548815 | 1.546456     | 4.799564e+07 |
| 512  | 1.567435 | 1.554045 | 1.551760 | 1.546456     | 5.747376e+07 |
| 1024 | 1.568899 | 1.549483 | 1.547350 | 1.546456     | 9.018599e+07 |

|      |          |          |          |          |              |
|------|----------|----------|----------|----------|--------------|
| 2048 | 1.565558 | 1.549502 | 1.547294 | 1.546456 | 4.886845e+07 |
| 4096 | 1.564763 | 1.549069 | 1.546713 | 1.546456 | 6.001418e+07 |

### Task c

```
Logistic Regression with Interaction Coefficients: 0.05433160780385303  
-1.9818073563595187 0.9257953193089048 0.2769188600543171  
Original Coefficients: 0.1, -2, 1, 0.2
```

We can see that model got very close to the original coefficients used to generate the data.

Probabilistic models - All

Discriminative models:

- LR
- LRi

Generative models: - NB

With small  $n$ , Naive Bayes often outperforms logistic regression (same pattern as Ng & Jordan 2001). With large  $n$ , logistic regression with the interaction term dominates, and approaches the performance of Optimal Bayes. Perplexity (log-loss) highlights probability quality; models without the interaction term score worse in perplexity even when classification accuracy looks ok.

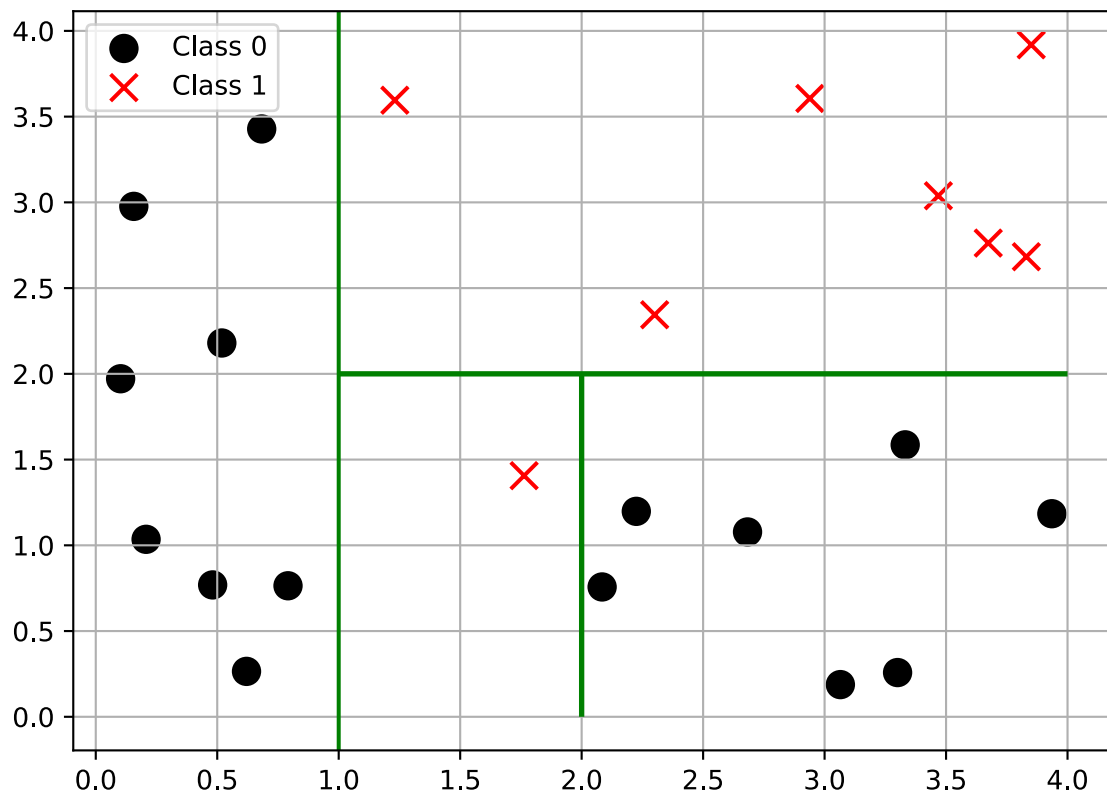
NB learns parameters faster from small datasets (higher bias, low variance). LR converges more slowly but achieves better asymptotic performance.

A model that includes this interaction matches the true function class, so as more data come in, it converges to the true parameters and thus becomes nearly optimal.

Dummy may beat LR without interaction when  $n$  is very small, because LR without interaction overfits or is too uncertain (not the case in this problem). Same for Optimal Bayes when  $n$  is very small some model can get “lucky” with few samples and outperform it but in general Optimal Bayes is the best as it uses the true data generating process.

### Problem 13

Dataset was taken from .rmd file provided in the exercise. The



Dataset was taken from .rmd file provided in the exercise. The splits created by the given decision tree is shown in green and it was not computed but chosen by hand. The Gini Coefficient for each split is can be found below.

```
def gini(side):
    if len(side)==0: return 0
    p = np.mean(side==1)
    return 2*p*(1-p)

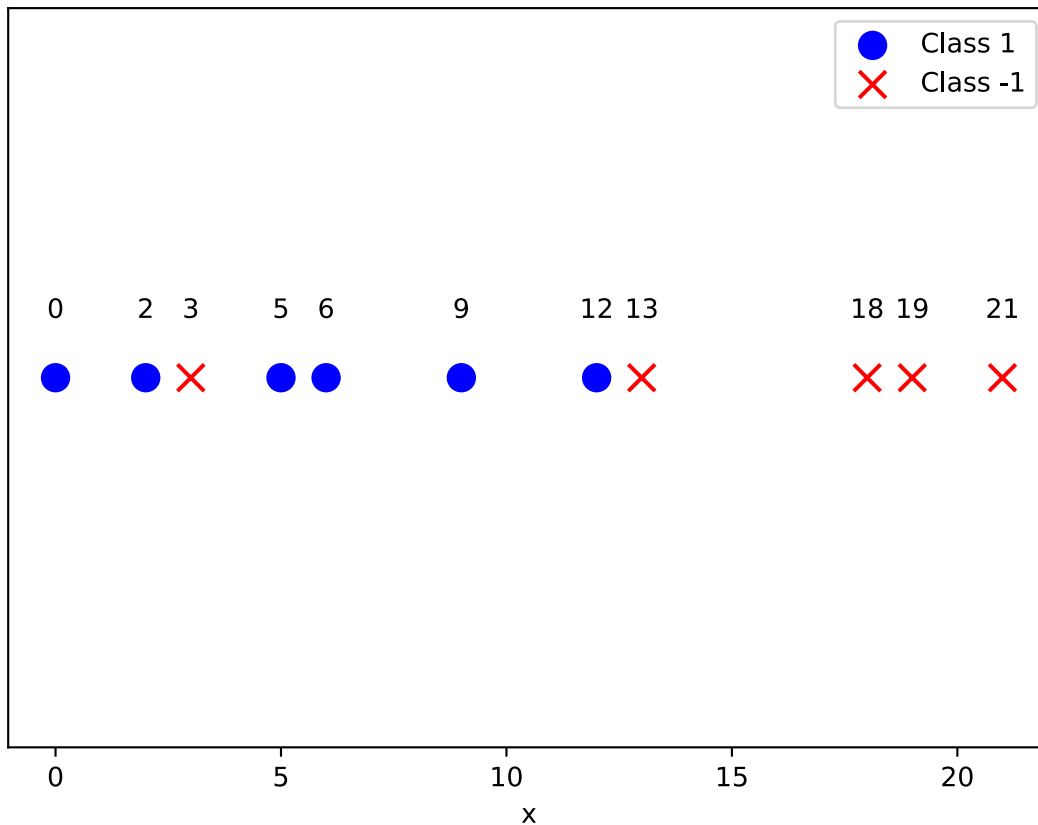
def split_gini(mask):
    left = Y[mask]
    right = Y[~mask]
    n = len(Y)
    return (len(left) / n) * gini(left) + (len(right) / n) * gini(right)
```

```
Gini for first split: 0.32463768115942027
Gini for second split: 0.07608695652173914
Gini for third split: 0.0
```

## Problem 14

### Task a

Firstly lets visualize the dataset.



#### 1-NN

Blue class (1) boundry  $(-\infty, 2.5) \cup (4, 12.5)$

Red class (-1) boundry  $(2.5, 4) \cup (12.5, \infty)$

#### 3-NN

Blue class (1) boundry  $(-\infty, 13.5)$

Red class (-1) boundry  $(13.5, \infty)$

As 13 should be classified as 1 in 3-NN (13, 12, 9 are the closest neighbors) and 18 should be classified as -1 in 3-NN (18, 19, 21 are the closest neighbors). Near 13 13 and 12 are always the closest neighbors so we need to find the point where 18 becomes closer than 9. This happens at point  $(18 + 9)/2 = 13.5$ .

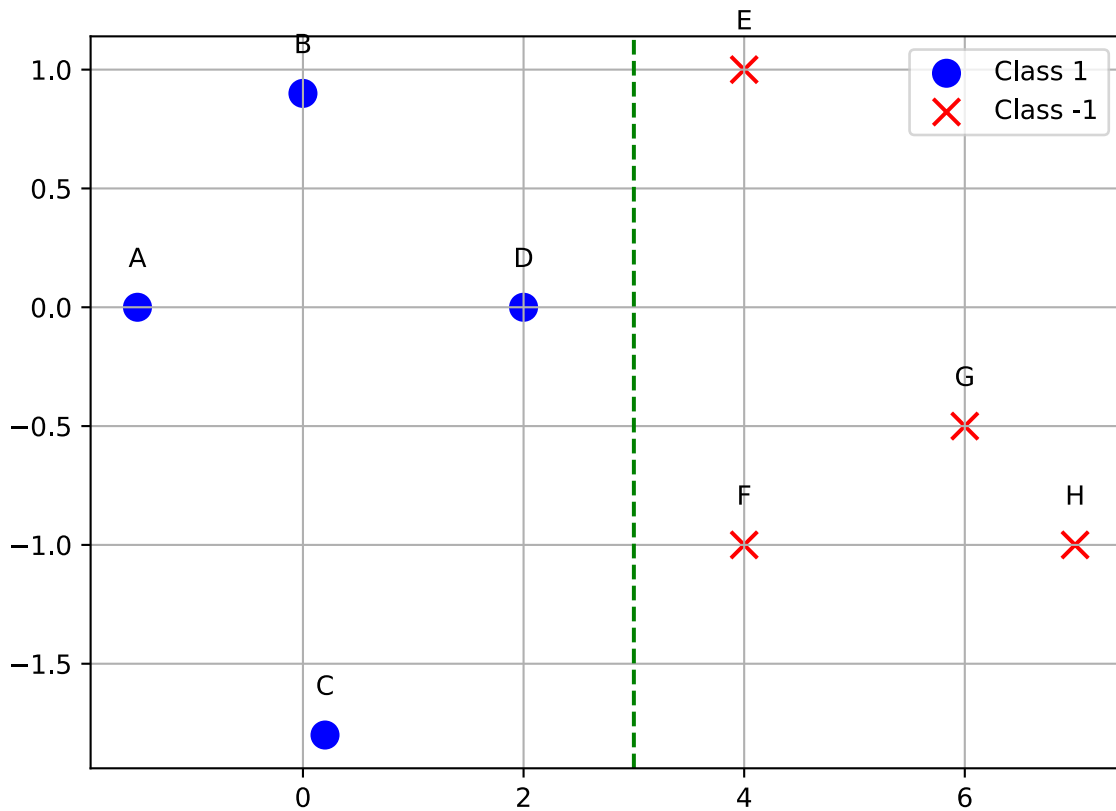
### Task b

In general, the bigger the  $k$  the smoother the decision boundary (less complex). With large  $k$  the model is less sensitive to noise but might underfit the data. With small  $k$  the model is more sensitive to noise and might overfit the data.

Examples: with our dataset 1-NN overfits the data (there is an extra interval) while 13-NN would underfit the data (classifies all points as 1).

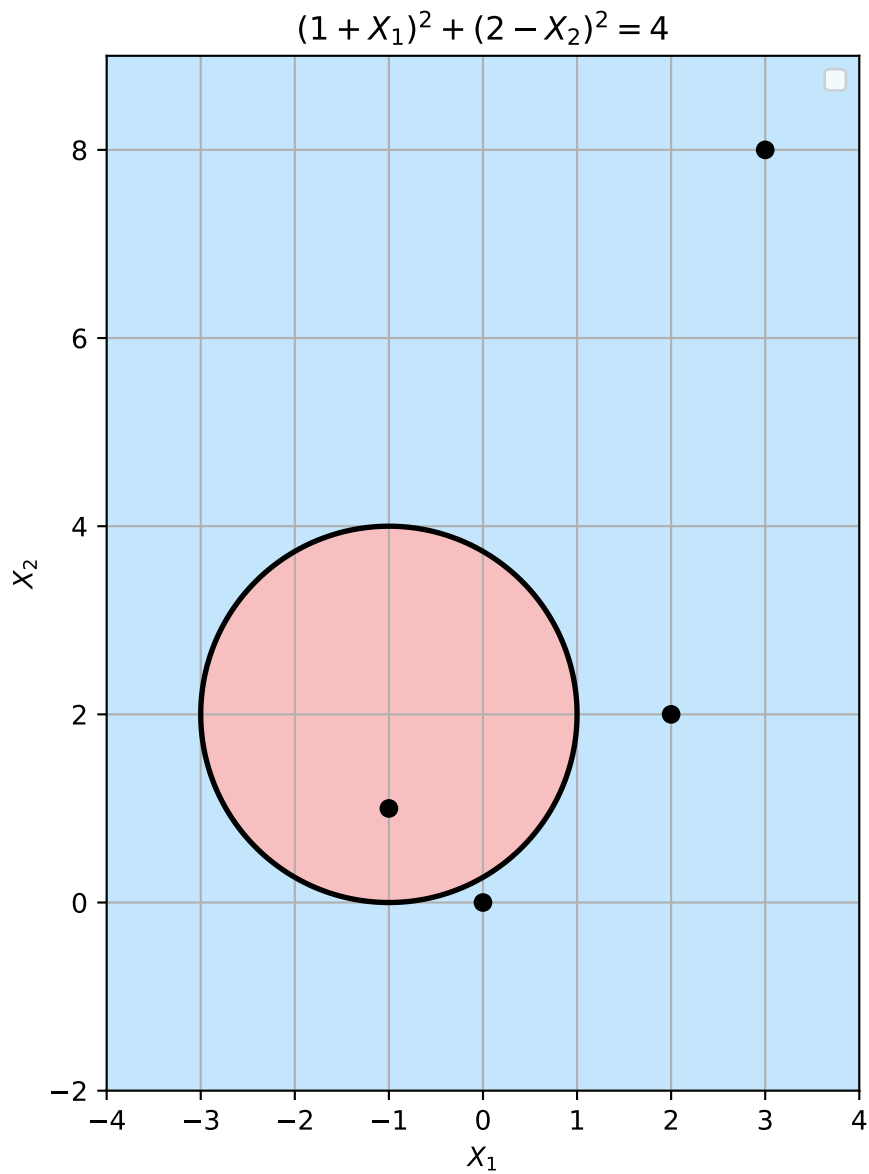
## Problem 15

### Task a



$x_1 - 3 = 0$  is equation for the hyperplane. Margin is 1.0 as the closest points to the hyperplane are D (2,0) and E (4,1) with distance 1.0. Support vectors are points D, E and F.

### Task b and c



### Task c

$$(1 + x_1)^2 + (2 - x_2)^2 = x_1^2 + x_2^2 + 2x_1 - 4x_2 + 5 = 4$$

if we assign  $Z_1 = x_1$ ,  $Z_2 = x_2$ ,  $Z_3 = x_1^2$ ,  $Z_4 = x_2^2$  we get

$$Z_3 + Z_4 + 2Z_1 - 4Z_2 + 1 = 0$$

which is a linear equation in the new feature space.